PROJECT 8 | SNOWFLAKE CAPSTONE | INSURANCE, CLAIMS & FRAUD

ClaimGuard 360

A Self-Starting, Self-Checking and Self-Healing Insurance Data Pipeline

AWS S3 → IDMC → Bronze → Silver → Gold → Snowpark → Streamlit

Stand-out features added on top of the base project
1. IDMC File Listener – the pipeline starts by itself when files arrive
2. Control File Check – proves no record was lost or added in transit
3. Reprocessing of Rejected Files – fixed records come back in automatically
4. Threshold Alerts – the platform warns the business before problems grow
5. Region-Based Access – each analyst sees only their own region's data

7 source files • 6,080 records • 5 days • 5 trainees

1. The Idea in Simple Words

An insurance company receives seven files every cycle: customers, insurance products, policies, premium payments, claims, claim settlements and fraud cases. Today, nobody can easily answer simple questions such as *"How much premium did we collect?"*, *"Do settlements match approved claims?"* or *"Which claims look suspicious?"*.

ClaimGuard 360 solves this by moving the files from AWS S3 through a cleaning step (IDMC) and three quality layers in Snowflake (Bronze, Silver, Gold), then scoring fraud risk with Snowpark and showing everything on a Streamlit dashboard.

> **What makes this project different?**
>
> Most capstone pipelines must be **started by hand**, **trust the file blindly**, **throw bad records away**, **wait for someone to look at a dashboard** and **show everyone everything**. ClaimGuard 360 does the opposite: it **starts itself** (File Listener), **verifies every file** (Control File), **gives rejected records a second chance** (Reprocessing), **raises its own alarms** (Threshold Alerts) and **protects data by region** (Region-Based Access).

Base project vs ClaimGuard 360

Question	Typical capstone	ClaimGuard 360
How does the run start?	Someone clicks Run	File Listener starts it when files land
How do we know the file is complete?	We assume it is	Control file count and total amount are matched
What happens to rejected rows?	They sit in a reject folder forever	Tracked, fixed, resubmitted and closed
How is the business warned?	Only if someone opens the dashboard	Automatic email when a limit is crossed
Who sees which data?	Role decides tables only	Role + region decides rows; PII is masked

2. The Data at a Glance

File	Records	What it holds
customers.csv	800	Customer details, contact, city, state, **region**, risk category
insurance_products.csv	80	Product type, coverage, sum insured limits, grace period
policies.csv	1,500	Which customer bought which product, dates, premium, status
premium_payments.csv	1,400	Payments received against policies
claims.csv	1,200	Claims raised, loss date, claimed and approved amount
claim_settlements.csv	700	Gross, deductible, recovery and net amount paid
fraud_cases.csv	400	Investigations, fraud score, outcome
Total	**6,080**	

Planted quality problems the pipeline must catch: **8** null values, **3** duplicates, **3** business anomalies, **2** outliers and **4** different date formats.

3. The Complete Flow – Step by Step

The table below is the full journey. Each step is then explained in simple terms in the pages that follow. Stand-out features are marked with ★.

#	Step	One-line meaning
1	Organise S3 folders	Give every file a clear place to land, wait, fail or retire
2	★ File Listener	Watch S3 and start the pipeline automatically
3	★ Control File Check	Confirm the file is complete before touching it
4	IDMC Cleaning & Routing	Fix formats, catch duplicates, sort good from bad
5	★ Reject Reprocessing	Fix and resend rejected records; track until closed
6	Connect Snowflake to S3	Secure, password-free link to read clean files
7	Bronze Layer	Raw copy of the file plus audit details
8	Silver Layer	Correct types, latest version, valid relationships
9	Gold Layer	Dimensions, facts, marts and the 10 KPIs
10	Incremental Claims (Stream + Task)	Process only new or changed claims, safely
11	Snowpark Fraud Score	Explainable Low / Medium / High risk for each claim
12	Security + ★ Region Access	Roles, masking, secure views and row-level region filter
13	★ Threshold Alerts	Platform emails the business when a limit is crossed
14	Streamlit Dashboard	Portfolio, Claims, Fraud and Pipeline Health tabs
15	Deploy, Test & Recover	Run in order, prove results, undo mistakes

Remember it as one sentence

Files land in S3 → the **listener** wakes up → the **control file** confirms the file is whole → IDMC **cleans and sorts** → bad rows go to **reprocessing** → Bronze **keeps** → Silver **corrects** → Gold **summarises** → Snowpark **scores** → security **filters by role and region** → alerts **warn** → Streamlit **shows**.

4. Each Step Explained

Step 1: Organise the S3 Folders

What happens: Before anything runs, the S3 bucket is arranged so every file has a clear home at each stage of its life.

What is inside:

- **landing/** – one sub-folder per source (customers, policies, claims …). New files arrive here.
- **control/** – the small control file that travels with each data file.
- **clean/, warning/, duplicate/, reject/** – where IDMC places sorted records.
- **resubmit/** – where corrected rejected records are dropped for another try.
- **archive/yyyy/mm/dd/** – processed files are moved here, never deleted.
- **quarantine/** – files that fail the control check are parked here.

Why it matters: Anyone can look at the bucket and instantly know the state of every file.

Evidence to show: *Screenshot of the S3 folder tree.*

Step 2: IDMC File Listener ★

What happens: The pipeline is not started by a person. An IDMC File Listener keeps watching the S3 landing folder and starts the Taskflow on its own as soon as the expected files arrive.

What is inside:

- Watches the **landing/** folder using the S3 connection.
- Reacts to a **file-arrival** event matching a name pattern such as *claims_*.csv*.
- Can wait for a **"ready" signal** (the control file) so it does not start on a half-uploaded file.
- Triggers the IDMC **Taskflow**, which runs the mappings in order: masters → policies → payments and claims → settlements and fraud.
- Every trigger is written to an audit table with the file name and time.

Why it matters: Removes manual effort and human delay. The data is ready minutes after it arrives, at any hour.

Evidence to show: *Upload a claims file live during the demo and show the Taskflow starting by itself.*

Step 3: Control File Check ★

What happens: Every data file comes with a tiny partner file (for example *claims_20261103.ctl*) that says what the data file *should* contain. The pipeline compares the promise with the reality before doing any work.

What is inside:

- The control file holds: **file name, expected row count, total of a key amount** (e.g. sum of claim_amount) and **business date**.
- IDMC counts the actual rows and adds up the actual amount.
- **Match** → the file continues to cleaning.
- **Mismatch** → the file is moved to **quarantine/**, the run stops for that source and an alert is raised.
- The result (expected vs actual) is saved in a CONTROL table for audit.

Why it matters: It proves no record was lost, cut off or added between the source system and the platform. This is how real banks and insurers work.

Evidence to show: *Edit a control file to a wrong count and show the file going to quarantine.*

Step 4: IDMC Cleaning and Routing

What happens: IDMC reads each verified file, fixes what can be fixed and sorts every record into the right bucket.

What is inside:

- **Standardise dates** – the 4 different date formats are converted into one format and kept.
- **Optional nulls** (8) – kept, but marked as **Warning**.
- **Duplicates** (3) – sent to the **Duplicate** route.
- **Impossible records** (3) – sent to **Reject** with a clear reason: policy expires before it starts, loss happens before the policy starts, or net settlement ≠ gross – deductible – recovery.
- **Extreme but possible** (2) – the ■25,000,000 claim and the fraud score of 145 are kept and marked as **Warning**.
- Every reject row carries a **reject_reason** and a **reject_id** so it can be fixed later.

Why it matters: Bad data never silently reaches the reports, and every decision is explainable.

Evidence to show: *Counts per route matching the expected 8 / 3 / 3 / 2 / 4.*

Step 5: Reprocessing of Rejected Files ★

What happens: Rejected records are not a dead end. They are tracked like tickets until they are fixed and loaded, or formally closed.

What is inside:

- Each rejected row is logged in a **Reject Register** with its reason and status **OPEN**.
- The data owner corrects the row and drops the file into **resubmit/**.
- The File Listener sees the new file and runs it through the same checks again; status becomes **RESUBMITTED**.
- If it now passes, it flows on to Bronze and the status becomes **RESOLVED**.
- If it fails again, the attempt count increases. After 3 attempts the status becomes **CLOSED – NOT FIXABLE**.
- The dashboard shows how many rejects are open, fixed and closed.

Why it matters: The business does not lose genuine claims because of a typo, and auditors can see exactly what happened to every bad record.

Evidence to show: *Fix the 'loss date before policy start' record, resubmit it and show it moving from OPEN to RESOLVED.*

Step 6: Connect Snowflake to S3

What happens: Snowflake needs a safe way to read the clean files from S3.

What is inside:

- A **storage integration** – a trusted link between Snowflake and an AWS role; no passwords are kept in scripts.
- A **file format** – tells Snowflake how to read the CSV (header, quotes, empty values).
- An **external stage** – a named pointer to the **clean/** folder.

Why it matters: Secure and repeatable access. Note: the base document uses an internal stage; ClaimGuard 360 uses an external S3 stage because the source is S3.

Evidence to show: *LIST command showing the clean files visible from Snowflake.*

Step 7: Bronze Layer – Keep Everything as It Came

What happens: The clean files are loaded into Snowflake exactly as they are.

What is inside:

- All columns are stored as text so nothing fails during loading.
- Extra columns record the **file name, row number, load time and batch ID**.
- Column order matches the document exactly (an acceptance rule).
- A load audit table compares **rows in file vs rows loaded vs rows failed**.
- Only engineering roles can see Bronze.

Why it matters: You can always trace any number back to the original file and row.

Evidence to show: *Audit table showing 6,080 records accounted for.*

Step 8: Silver Layer – Make the Data Trustworthy

What happens: Raw text is turned into correct, reliable records.

What is inside:

- Text becomes real **dates, numbers and true/false** values.
- If a record appears more than once, only the **latest version** (by source_updated_ts) is kept.
- **Relationship checks**: every policy has a customer and product, every claim has a policy and customer, every settlement and fraud case has a claim.
- Orphan records go to an exceptions table instead of disappearing.
- Each row carries a **quality status** and **reason**.

Why it matters: Gold and the dashboard are only built from data you can defend.

Evidence to show: *Bronze-to-Silver reconciliation report.*

Step 9: Gold Layer – Business-Ready Data

What happens: Data is shaped for analysis and the KPIs are calculated.

What is inside:

- **Dimensions** (descriptions): DIM_CUSTOMER, DIM_INSURANCE_PRODUCT, DIM_POLICY, DIM_DATE.
- **Facts** (events and amounts): FACT_PREMIUM_PAYMENT, FACT_CLAIM, FACT_CLAIM_SETTLEMENT, FACT_FRAUD_CASE.
- **Marts** (ready summaries): MART_POLICY_PORTFOLIO, MART_CLAIMS_PERFORMANCE, MART_SETTLEMENT_RECONCILIATION, MART_FRAUD_RISK.
- **10 KPIs**: active policies and sum insured, premium collected, claim frequency and average claim, approval ratio, settlement turnaround time, net settlement amount, loss ratio proxy, open fraud investigations, confirmed fraud rate, high-risk claims.
- Region is carried from the customer into every mart so region filtering works later.

Why it matters: Business users get one consistent version of the truth.

Evidence to show: *KPI validation sheet comparing Gold numbers with manual checks.*

Step 10: Incremental Claims – Stream and Task

What happens: When new or corrected claims arrive, only the changes are processed instead of reloading everything.

What is inside:

- A **Stream** remembers which claim rows are new in Bronze.
- A **Task** runs on a schedule, but only if the Stream has data. It inserts new claims and updates corrected ones (only if the new version is newer).
- Follow-on Tasks refresh the **Gold marts**, then the **fraud score**, then the **alert checks**.
- **Idempotent replay**: loading the same file twice changes nothing.

Why it matters: Fast, cheap and safe updates – exactly what the document asks to test.

Evidence to show: *Initial load → second file → replay; row counts before and after.*

Step 11: Snowpark Fraud Risk Score

What happens: A Python program running inside Snowflake gives each claim an explainable risk rating.

What is inside:

- **Amount points** – very large claims score higher.
- **Early-claim points** – a claim soon after the policy started scores higher.
- **History points** – customers with many earlier claims score higher.
- **Investigation points** – past suspicious or confirmed fraud cases add points.
- Points are added to a **total score** → **Low / Medium / High**.
- Each part of the score, the total, the category and the time are stored, so anyone can see *why* a claim is High.

Why it matters: Transparent, repeatable scoring – no black-box machine learning needed.

Evidence to show: *Same input, same score on two runs; drill-down of one High claim.*

Step 12: Security and Region-Based Access

What happens: Every user sees only what they should – by role *and* by region.

What is inside:

- **Roles**: INSURANCE_ANALYST_ROLE, CLAIMS_ANALYST_ROLE, FRAUD_MANAGER_ROLE.
- **Masking**: email, phone and address are hidden from roles that do not need them.
- **Secure views**: reports read from protected views, never from raw tables.
- ★ **Region filter (row access policy)**: a small mapping table links each user or role to one or more regions (e.g. North, South). When an analyst queries, Snowflake automatically returns only rows from their regions. Fraud managers can see all regions.
- The same rule applies in Streamlit, because Streamlit reads the same secure views.
- Every role is tested with USE ROLE.

Why it matters: Customer data stays private, and regional teams get focused, relevant views – as in real insurers.

Evidence to show: *Same query run as a South analyst and a North analyst showing different results.*

Step 13: Threshold Alerts ★

What happens: The platform watches key numbers and emails the business when they cross a limit – no one has to keep checking the dashboard.

What is inside:

- Limits are stored in a simple **threshold table**, so they can be changed without touching code.
- Example limits: **High-risk claims** above 20 in a day; **loss ratio proxy** above 70%; **single claim** above ■1 crore; **open fraud investigations** older than 30 days; **control-file mismatch**; **late file** not arrived by 9 AM.
- A Snowflake **Alert** checks the condition on a schedule (or after each Gold refresh).
- When triggered, an email is sent and a row is written to an **alert history** table.
- The dashboard shows active and past alerts.

Why it matters: The pipeline becomes proactive – it tells the business about risk instead of waiting to be asked.

Evidence to show: *Load a claim above the limit and show the alert email and history row.*

Step 14: Streamlit Dashboard

What happens: A web app inside Snowflake presents everything in one place.

What is inside:

- **Portfolio tab** – active policies, sum insured, premium collected.
- **Claims tab** – claim frequency, approval ratio, settlement turnaround, net settled.
- **Fraud tab** – Low/Medium/High mix, high-risk list, click a claim to see its score breakdown.
- ★ **Pipeline Health tab** – file arrivals, control check results, reject register (open / resolved / closed), alert history.
- A **reconciliation panel** proves dashboard totals equal Gold totals.
- Region filtering happens automatically based on who is logged in.

Why it matters: Business users get answers; reviewers see the pipeline is healthy.

Evidence to show: *Live demo with two different roles.*

Step 15: Deploy, Test and Recover

What happens: The whole platform can be rebuilt and rerun in a fixed, safe order.

What is inside:

- All scripts kept in **Git**, numbered in run order (setup → stages → Bronze → Silver → Gold → stream/task → Snowpark → security → alerts → Streamlit).
- IDMC assets exported and stored with the scripts.
- **Zero-Copy Clone** of Gold before big reruns, as a free backup.
- **Time Travel** to undo a bad load.
- A **runbook** explains how to start, stop, rerun and recover.

Why it matters: Anyone in the team can deploy and support it – not only the person who built it.

Evidence to show: *Drop a Gold table, restore it with Time Travel during the demo.*

5. Control Tables that Power the Stand-Out Features

These small tables live in the CONTROL schema. They are what turn a normal pipeline into a self-managing one.

Table	Used by	What it stores
FILE_ARRIVAL_LOG	File Listener	File name, source, arrival time, Taskflow run ID
CONTROL_FILE_CHECK	Control File Check	Expected vs actual rows and amount, MATCH / MISMATCH
LOAD_AUDIT	Bronze load	Rows in file, loaded, failed, batch ID
DQ_EXCEPTIONS	Silver	Orphans and failed rules with reason
REJECT_REGISTER	Reprocessing	Reject ID, reason, attempts, status (OPEN / RESUBMITTED / RESOLVED / CLOSED)
ALERT_THRESHOLDS	Threshold Alerts	Metric name, limit, recipients, on/off
ALERT_HISTORY	Threshold Alerts	When, which metric, value, message sent
USER_REGION_MAP	Region Access	User or role and allowed regions

6. The Five Stand-Out Features – Quick Reference

Feature	Problem it solves	How it works (simple)	Demo moment
IDMC File Listener	Pipeline waits for someone to press Run	Watches S3, starts the Taskflow when files arrive	Upload a file → run starts by itself
Control File Check	Incomplete or tampered files go unnoticed	Compare expected count and amount with actual	Wrong count → file quarantined
Reject Reprocessing	Genuine records lost due to small errors	Register → fix → resubmit → recheck → resolve	OPEN changes to RESOLVED
Threshold Alerts	Risks seen only when someone checks	Scheduled checks against a limit table, then email	Big claim → email arrives
Region-Based Access	Analysts see data they should not	Row access policy using a user-to-region map	Two users, two different results

7. Five-Day Plan with the Extra Features

Day	Base work (from the document)	Extra ClaimGuard 360 work
Day 1	Discovery, profiling of 7 files, keys, grain, KPIs, mappings	Design S3 folders, control file layout, threshold list and region map
Day 2	IDMC cleansing and routing, stages, formats, Bronze, audit	Build File Listener, control file check and reject register
Day 3	Silver, dimensions, facts, marts, KPIs, reconciliation	Carry region into marts; build resubmit flow
Day 4	Stream/Task, second load, replay, roles, masking, Snowpark, recovery	Row access policy by region; threshold alerts and alert history
Day 5	Streamlit, end-to-end testing, UAT, runbook, demo	Pipeline Health tab; rehearse the live demo

8. Suggested Live Demo Script (10 minutes)

1. Show the empty S3 landing folder and the dashboard Pipeline Health tab.
2. Upload the claims file and its control file – the File Listener starts the Taskflow on its own.
3. Show the control check result: expected 1,200 = actual 1,200 → MATCH.
4. Show the routing counts: warnings, duplicates and rejects exactly as planned.
5. Fix one rejected record, drop it in resubmit/ – watch the register change to RESOLVED.
6. Load the second claims file – the Stream and Task update only the changed claims; replay changes nothing.
7. Open the Fraud tab and drill into one High-risk claim to explain its score.
8. Show the threshold alert email triggered by the ■25,000,000 claim.
9. Log in as a regional analyst – only their region appears, and contact details are masked.
10. Drop a Gold table and restore it using Time Travel.

9. Testing Checklist

Test	Expected result
All 7 files processed	6,080 records accounted for across all routes
File Listener	Taskflow starts within minutes of arrival; logged
Control file mismatch	File quarantined, alert raised, no load
DQ routing	8 nulls, 3 duplicates, 3 anomalies, 2 outliers, 4 date formats handled
Reject reprocessing	Fixed record loads; status RESOLVED; 3 failures → CLOSED
Relationships	No orphan reaches Gold
KPIs	Gold values match manual calculation
Incremental and replay	New + corrected claims applied; replay leaves counts unchanged
Snowpark score	Same result on rerun; components stored
Roles, masking, region	Each role sees correct columns and only its regions
Threshold alerts	Alert fires once per breach and is logged
Streamlit vs Gold	Totals match

Test	Expected result
Recovery	Time Travel / clone restores data

10. Simple Glossary

Term	Meaning in one line
Bronze / Silver / Gold	Raw copy → cleaned and trusted → business-ready summaries
File Listener	An IDMC watcher that starts a job when a file appears
Control file	A small file stating how many rows and what total the data file should have
Stream	Snowflake's memory of which rows are new since last time
Task	A scheduled job inside Snowflake
Idempotent	Running the same load twice gives the same result
Row access policy	A rule that hides rows a user is not allowed to see
Masking policy	A rule that hides parts of a column, such as an email
Snowflake Alert	A scheduled check that acts when a condition is true
Time Travel	Snowflake's ability to see or restore data from the past